

LangChain Callback 系统架构

ReAct Loop 时间线映射与观测设计

技术架构白皮书 · 2026年6月

执行摘要

LangChain 采用 Mixin-Based 的分层回调设计，将事件观测深度集成到 Agent 执行生命周期中。本文档系统性地解析了 BaseCallbackHandler 的完整回调事件、在 ReAct Loop 中的触发时机、以及与 Pi 三层 Hook 系统的差异。通过对比分析，揭示了 LangChain Callbacks 的观测者本质与设计权衡。

一、回调系统架构

1.1 Mixin-Based 分层设计

LangChain 采用 **Mixin-Based** 分层设计，将回调按组件类型分层：

```
BaseCallbackHandler
|-- LLMManagerMixin      (on_llm_start, on_llm_new_token, on_llm_end, on_llm_error)
|-- ChainManagerMixin   (on_chain_start, on_chain_end, on_chain_error)
|-- ToolManagerMixin    (on_tool_start, on_tool_end, on_tool_error)
|-- RetrieverManagerMixin (on_retriever_start, on_retriever_end, on_retriever_error)
```

```
|-- CallbackManagerMixin      (整合调度)
|-- RunManagerMixin          (on_text, on_retry, on_custom_event)
```

关键设计原则：

- 关注点分离: 每个 Mixin 处理特定组件类型
- 生命周期钩子: 每个组件都有 start 、 end 和 error 回调
- 继承链: Handler 可以继承多个 Mixin 来组合行为
- 异步支持: 通过单独的 AsyncCallbackHandler 支持异步操作

1.2 完整回调事件列表

组件	开始	流式	结束	错误
LLM	on_llm_start	on_llm_new_token	on_llm_end	on_llm_error
Chat Model	on_chat_model_start	on_llm_new_token	on_llm_end	on_llm_error
Chain	on_chain_start	-	on_chain_end	on_chain_error
Tool	on_tool_start	-	on_tool_end	on_tool_error
Retriever	on_retriever_start	-	on_retriever_end	on_retriever_error
Agent	on_agent_action	-	on_agent_finish	(通过 chain/tool)
Retry	on_retry	-	-	-
Custom	on_custom_event	-	-	-

1.3 回调方法签名

```
class BaseCallbackHandler:
    raise_error: bool = False    # 回调错误时是否抛出异常
    run_inline: bool = False     # 是否内联执行

    def on_llm_start(self, serialized, prompts, run_id=None, **kwargs):
        """LLM 开始调用时触发"""

    def on_llm_new_token(self, token, *, chunk=None, run_id=None, **kwargs):
        """每个 Token 生成时触发 (流式) """

    def on_llm_end(self, response, *, run_id=None, **kwargs):
        """LLM 完成时触发"""
```

```
def on_tool_start(self, serialized, input_str, *, run_id=None, **kwargs):
    """工具开始执行时触发"""

def on_tool_end(self, output, *, run_id=None, **kwargs):
    """工具执行完成时触发"""

def on_agent_action(self, action, *, run_id=None, **kwargs):
    """Agent 执行动作时触发"""

def on_agent_finish(self, finish, *, run_id=None, **kwargs):
    """Agent 完成时触发"""
```

二、ReAct Loop 时间线映射

● 1. 整轮循环开始

on_chain_start

● 2. LLM 请求准备 (Reason Phase)

on_llm_start

on_llm_new_token × N

on_llm_end

● 3. Agent 动作解析 (Act Phase)

on_agent_action

on_tool_start

on_tool_end

on_tool_error

● 4. 观察阶段 (Observation Phase)

(隐式: 工具结果添加到上下文)

● 5. 循环决策 / 完成

on_agent_finish

on_chain_end

2.1 完整时间线示例

```
# 第一轮 ReAct 循环
on_chain_start(serialized={...}, inputs={...})
|-- on_llm_start(prompts=[...])
|-- on_llm_new_token("Let")
|-- on_llm_new_token(" me")
|-- on_llm_new_token(" search")
|-- on_llm_end(response=LLMResult(...))
```

```
|-- on_agent_action(action=AgentAction(tool="search", ...))
|-- on_tool_start(serialized={"name": "search"}, input_str="...")
|-- on_tool_end(output="Search results...")

# 第二轮循环 (如需要)
|-- on_llm_start(prompts=[...])
|-- on_llm_new_token("Now")
|-- on_llm_end(response=LLMResult(...))

|-- on_agent_action(action=AgentAction(tool="read", ...))
|-- on_tool_start(serialized={"name": "read"}, input_str="...")
|-- on_tool_end(output="File contents...")

# 完成
on_agent_finish(finish=AgentFinish(output="Final answer...", log="..."))
on_chain_end(outputs={...})
```

三、Callback 能力与限制

3.1 Callback 能做什么

能力	说明	示例
观测	读取所有事件数据	记录 Token、工具调用、错误
计数	统计 Token 使用量、调用次数	计费、性能分析
日志	记录执行流程	写入文件、发送到追踪系统
流式推送	实时推送事件到客户端	WebSocket、SSE
状态跟踪	维护运行状态	追踪 run_id、parent_run_id
条件跳过	通过 ignore_* 属性跳过事件	ignore_llm = True

3.2 Callback 不能做什么

限制	说明	原因
修改输入	无法修改 prompts、messages	回调是只读观测器
修改输出	无法修改 LLM 响应、工具结果	执行已完成，无法回溯
阻断执行	无法阻止工具调用或 LLM 请求	回调无返回值语义

改变流程	无法改变 Agent 决策	回调不参与控制流
修改上下文	无法修改消息历史	上下文是不可变的
拦截流	无法拦截或修改流式 Token	流式数据已发出

3.3 关键设计特性

```
def handle_event(handlers, event_name, ignore_condition_name, *args, **kwargs):
    coros = []
    for handler in handlers:
        try:
            # 1. 忽略条件检查
            if not getattr(handler, ignore_condition_name, False):
                event = getattr(handler, event_name)(*args, **kwargs)
                # 2. 异步聚合
                if asyncio.iscoroutine(event):
                    coros.append(event)
        except NotImplementedError:
            # 3. 回退链: on_chat_model_start -> on_llm_start
            if event_name == "on_chat_model_start":
                handle_event([handler], "on_llm_start", ...)
        except Exception as e:
            # 4. 错误隔离
            logger.warning("回调错误: %s", e)
            if handler.raise_error:
                raise
```

关键设计模式：

- 忽略条件: Handler 可以声明 `ignore_*` 属性来跳过事件
- 回退链: `on_chat_model_start` 回退到 `on_llm_start`
- 异步聚合: 收集协程并适当执行
- 错误隔离: 一个 handler 的错误不会破坏其他 handler

四、代码示例

4.1 自定义回调处理器

```
from langchain_core.callbacks.base import BaseCallbackHandler
from langchain_core.outputs import LLMResult

class TokenCountingHandler(BaseCallbackHandler):
    """统计所有 LLM 调用的 Token 数量"""

    def __init__(self):
        self.total_tokens = 0
```

```

        self.prompt_tokens = 0
        self.completion_tokens = 0

    def on_llm_end(self, response: LLMResult, **kwargs) -> None:
        for generation in response.generations:
            for gen in generation:
                if gen.generation_info:
                    usage = gen.generation_info.get('token_usage', {})
                    self.prompt_tokens += usage.get('prompt_tokens', 0)
                    self.completion_tokens += usage.get('completion_tokens', 0)
                    self.total_tokens += usage.get('total_tokens', 0)

    def on_llm_error(self, error, **kwargs) -> None:
        print(f"LLM 错误: {error}")

# 使用
handler = TokenCountingHandler()
llm = ChatOpenAI(callbacks=[handler])
result = llm.invoke("Hello!")
print(f"总 Token: {handler.total_tokens}")

```

4.2 流式回调

```

class StreamingHandler(BaseCallbackHandler):
    def on_llm_new_token(self, token: str, **kwargs) -> None:
        sys.stdout.write(token)
        sys.stdout.flush()

    def on_llm_start(self, serialized, prompts, **kwargs) -> None:
        print("\n[LLM 开始]")

    def on_llm_end(self, response, **kwargs) -> None:
        print("\n[LLM 结束]")

llm = ChatOpenAI(streaming=True, callbacks=[StreamingHandler()])

```

4.3 WebSocket 回调

```

class WebSocketCallbackHandler(BaseCallbackHandler):
    def __init__(self, websocket):
        self.websocket = websocket

    def on_llm_new_token(self, token, **kwargs):
        asyncio.create_task(self.websocket.send(token))

    def on_tool_start(self, serialized, input_str, **kwargs):
        asyncio.create_task(self.websocket.send({
            "type": "tool_start", "tool": serialized["name"]
        }))

    def on_tool_end(self, output, **kwargs):

```

```
asyncio.create_task(self.websocket.send({
    "type": "tool_end", "output": output
}))
```

五、关键设计原则总结

观测者模式: LangChain Callback 是一个单向的观测和监听框架, 设计用于:

- 追踪 Agent 执行流程
- 收集性能指标(Token 用量、延迟)
- 实时推送事件(WebSocket、SSE)
- 集成外部系统(LangSmith、Langfuse 等)

只读语义: 所有回调方法返回 `void`, 不修改任何执行状态。回调是被动观测者, 不是主动参与者。

错误隔离: `handle_event` 的 try-except 保证一个 handler 的崩溃不会破坏其他 handler 或主流程。

一句话总结: LangChain Callback 是执行后的观测系统——它告诉你"发生了什么", 但不能改变"将要发生什么"。